

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ
КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ОТЧЕТ ПО ИГРЕ

по дисциплине
«Информатика и программирование»

Студент		
гр. БИН-25-3	_____	Т. Г. Пономаренко
Ассистент		
преподавателя	_____	М.В. Водяницкий

Задание

Разработать консольную текстовую ролевую игру на языке Python. Программа должна работать в интерактивном пошаговом режиме, управляться командами с клавиатуры и предоставлять пользователю классический RPG-опыт.

Общая идея. Проект представляет собой текстовую RPG, реализованную в одном файле, в которой игрок:

- создаёт персонажа с выбором расы (Человек, Эльф, Дворф);
- исследует случайно генерируемое подземелье с различными комнатами;
- сражается с врагами, сложность которых растёт с каждым этажом;
- собирает предметы, экипирует оружие и броню;
- развивает персонажа, повышая уровень и распределяя очки характеристик.

Создание персонажа. В начале игры пользователь выбирает расу персонажа. Каждая раса определяет диапазоны генерации характеристик: здоровье, атака, защита, ловкость, а также рост и вес. Физические параметры влияют на игровые характеристики (например, высокий рост или большой вес снижают ловкость).

Прогрессия и экипировка. Персонаж получает опыт за победу над врагами. При накоплении опыта повышается уровень, давая очки для улучшения характеристик. Инвентарь позволяет хранить предметы, использовать зелья, экипировать оружие и броню, которые влияют на боевые параметры.

Подземелье и встречи. Игровой процесс построен на последовательном прохождении комнат. На развилках игрок выбирает направление, где его могут ждать враги, сокровища, комнаты отдыха или пустые помещения. Видимость содержимого комнат определяется случайным образом.

Боевая система. Сражения происходят в пошаговом режиме. На исход боя влияют характеристики персонажа и врага, экипировка, а также случайные факторы (уклонение, разброс урона). После победы игрок получает опыт, золото и с шансом 30% дополнительный предмет.

Содержание

Введение	3
1. Гейм-дизайн	4
2. Выполнение	5
2.1. Основная программа и главный цикл	5
2.2. Класс персонажа	5
2.3. Класс предметов	6
2.4. Класс врагов	7
2.5. Игровой менеджер	8
2.6. Боевая система	9
2.7. Система инвентаря	10
2.8. Генерация подземелья	11
3. Тестирование	12
Заключение	13

Введение

Разработка консольных приложений, в частности игр, является важным этапом в освоении программирования, так как позволяет применить на практике знания об алгоритмах, структурах данных, объектно-ориентированном проектировании и взаимодействии с пользователем.

Целью данной работы была реализация прототипа консольной текстовой ролевой игры (RPG) на языке Python, соответствующего заданным требованиям. Основной акцент делался на создании логичной, расширяемой архитектуры и отработке ключевых игровых механик, таких как создание и развитие персонажа, процедурная генерация контента и тактическая боевая система.

1 Гейм-дизайн

При проектировании игры были заложены следующие ключевые принципы, определившие архитектурные и программные решения:

- Простота и ясность управления: весь игровой процесс управляется через последовательность текстовых меню с числовым вводом, что делает интерфейс доступным и предсказуемым для пользователя. Игра реализована в одном файле, что упрощает её запуск и распространение.

- Ощущение прогрессии: игрок чувствует рост силы своего персонажа благодаря системе уровней, ручному распределению характеристик и получению более мощной экипировки по мере прохождения. Влияние роста и веса на ловкость добавляет дополнительный уровень реализма.

- Баланс выбора и случайности: благодаря процедурной генерации подземелья, врагов и наград каждая игровая сессия становится уникальной. При этом игрок сохраняет контроль над стратегическими решениями: выбор пути, тактика в бою, управление ресурсами инвентаря.

- Разнообразие игровых стилей: три доступные расы (Человек, Эльф, Дворф) имеют различные стартовые параметры и потенциальные пути развития. Эльфы отличаются высокой ловкостью, Дворфы — силой и выносливостью, а Люди имеют сбалансированные характеристики.

Эти принципы легли в основу проектирования модульной структуры программы, где каждый класс отвечает за определённую функциональность.

2 Выполнение

2.1 Основная программа и главный цикл

Вся игра реализована в одном файле `final.py`. Точкой входа является конструкция `if __name__ == "__main__":`, которая создаёт объект игры и запускает главное меню. Код запуска представлен на рисунке 1.

```
1 # ЗАПУСК ИГРЫ
2 if __name__ == "__main__":
3     print("Добро пожаловать в ТЕКСТОВУЮ RPG!")
4     game = Game()
5     game.main_menu()
6
```

Рисунок 1 – Листинг точки входа

Класс `Game` выступает в роли игрового менеджера, управляя всем процессом: от создания персонажа до исследования подземелья. Главный игровой цикл реализован в методе `play()`, который выполняется, пока персонаж жив.

2.2 Класс персонажа

Класс `Character` отвечает за создание и управление персонажем игрока. Конструктор класса, показанный на рисунке 2, принимает расу и генерирует характеристики в заданных диапазонах.

```

1 class Character:
2     def __init__(self, race):
3         self.race = race
4         self.level = 1
5         self.exp = 0
6         self.exp_to_next_level = 100
7         self.stat_points = 0
8
9         if race == "Человек":
10             self.hp_max = random.randint(80, 100)
11             self.attack = random.randint(10, 15)
12             self.defense = random.randint(8, 12)
13             self.agility = random.randint(10, 14)
14

```

Рисунок 2 – Листинг класса персонажа

После генерации характеристик вызывается метод `apply_size_modifiers()`, который корректирует ловкость в зависимости от роста и веса персонажа. Методы `get_total_attack()` и `get_total_defense()` вычисляют итоговые боевые параметры с учётом экипировки.

2.3 Класс предметов

Система предметов реализована через класс `Item` со статическим методом `generate_random_item()`. Код генератора предметов представлен на рисунке 3.

```

1 class Item:
2     @staticmethod
3     def generate_random_item():
4         item_types = [
5             {"name": "Малое зелье здоровья", "type": "potion", "
6             heal": 30},
7             {"name": "Среднее зелье здоровья", "type": "potion", "
8             heal": 50},
9             {"name": "Ржавый меч", "type": "weapon", "
10            attack_bonus": 2},
11            {"name": "Стальной меч", "type": "weapon", "
12            attack_bonus": 5},
13            {"name": "Кожаный доспех", "type": "armor", "
14            defense_bonus": 2}
15        ]
16        return random.choice(item_types)

```

Рисунок 3 – Листинг класса предметов

Предметы представлены словарями с различными типами: зелья (восстанавливают здоровье), оружие (увеличивают атаку), броня (увеличивает защиту) и золото.

2.4 Класс врагов

Класс Enemy отвечает за создание противников. Конструктор класса, показанный на рисунке 4, принимает номер этажа и генерирует характеристики с учётом прогрессии сложности.


```

1 class Enemy:
2     def __init__(self, floor):
3         self.names = ["Гоблин", "Скелет", "Орк", "Троль", "Зомби",
4             "Волк"]
5
6         self.name = random.choice(self.names)
7
8         multiplier = 1 + (floor - 1) * 0.3
9         self.hp = random.randint(30, 60) * multiplier
10        self.attack = random.randint(8, 15) * multiplier
11        self.defense = random.randint(5, 10) * multiplier
12        self.exp_reward = random.randint(20, 40) * multiplier

```

Рисунок 4 – Листинг класса врагов

Множитель сложности увеличивается с каждым этажом на 30%, что обеспечивает постепенное возрастание трудности. Метод `show_stats()` отображает характеристики врага перед боем.

2.5 Игровой менеджер

Класс `Game` является центральным компонентом игры, координирующим все остальные классы. Его конструктор и метод создания персонажа представлены на рисунке 5.

```

1 class Game:
2     def __init__(self):
3         self.player = None
4         self.current_floor = 1
5         self.rooms_cleared = 0
6         self.game_over = False
7         self.next_rooms = {"left": None, "right": None}
8
9     def create_character(self):
10        print("1 - Человек\n2 - Эльф\n3 - Дворф")
11        choice = input("Ваш выбор: ")
12        if choice == "1":
13            race = "Человек"
14        elif choice == "2":
15            race = "Эльф"
16        else:
17            race = "Дворф"
18        self.player = Character(race)
19

```

Рисунок 5 – Листинг игрового менеджера

Метод `main_menu()` отображает главное меню и запускает новую игру. Метод `play()` содержит основной игровой цикл.

2.6 Боевая система

Боевая система реализована в методе `battle()`. Код метода представлен на рисунке

6.

```

1 def battle(self, enemy):
2     while enemy.is_alive() and self.player.hp_current > 0:
3         print(f"Ваше HP: {self.player.hp_current}")
4         choice = input("1 - Атаковать: ")
5
6         if choice == "1":
7             damage = max(1, self.player.get_total_attack() -
8 enemy.defense)
9             enemy.hp -= damage
10            print(f"Вы нанесли {damage} урона!")
11
12            if not enemy.is_alive():
13                self.player.gain_exp(int(enemy.exp_reward))
14                self.player.coins += enemy.coin_reward
15                break

```

Рисунок 6 – Листинг боевой системы

Бой происходит в цикле, пока одна из сторон не погибнет. Урон рассчитывается как разница между атакой и защитой с гарантированным минимумом.

2.7 Система инвентаря

Управление инвентарём реализовано через методы `show_inventory()`, `use_item()` и `drop_item()`. Код метода использования предметов представлен на рисунке 7.

```

1 def use_item(self):
2     if not self.player.inventory:
3         print("Инвентарь пуст!")
4         return
5
6     self.show_inventory()
7     try:
8         choice = int(input("Выберите предмет: ")) - 1
9         item = self.player.inventory[choice]
10
11        if item["type"] == "potion":
12            self.player.hp_current += item["heal"]
13            self.player.inventory.pop(choice)
14            print(f"Вы восстановили {item['heal']} HP!")
15    except:
16        print("Неверный выбор!")
17

```

Рисунок 7 – Листинг системы инвентаря

Инвентарь хранится как список словарей в объекте персонажа. Для оружия и брони предусмотрены отдельные слоты экипировки.

2.8 Генерация подземелья

Подземелье генерируется процедурно с использованием методов `generate_room_type()`, `generate_specific_room()` и `show_paths()`. Код генерации комнат представлен на рисунке 8.

```

1 def generate_room_type(self):
2     room_types = ["enemy", "chest", "rest", "empty"]
3     weights = [40, 25, 20, 15]
4     return random.choices(room_types, weights=weights)[0]
5
6 def show_paths(self):
7     self.next_rooms["left"] = self.generate_room_type()
8     self.next_rooms["right"] = self.generate_room_type()
9
10    left_visible = random.random() < 0.6
11    if left_visible:
12        print(f"Слева: {self.get_room_name(self.next_rooms['left'])}")
13    else:
14        print("Слева: ???")
15

```

Рисунок 8 – Листинг генерации подземелья

Типы комнат генерируются с разными вероятностями: 40% — бой, 25% — сундук, 20% — отдых, 15% — пусто. Каждые 5 комнат происходит переход на новый этаж.

3 Тестирование

Для проверки корректности работы и сбалансированности игрового процесса было проведено ручное функциональное тестирование. Основные проверяемые сценарии включали:

- Создание персонажа: корректность генерации всех характеристик (здоровье, атака, защита, ловкость, рост, вес) в строго заданных для каждой расы диапазонах. Проверялось влияние роста и веса на ловкость.
- Система развития: правильность начисления опыта за победу над врагами, своевременное повышение уровня персонажа и начисление очков улучшения. Проверялась работа функции ручного распределения очков характеристик.
- Экипировка и инвентарь: корректность применения бонусов от надетого оружия и брони к характеристикам персонажа. Тестировались операции добавления предметов в инвентарь, их использования и выбрасывания.
- Генерация подземелья: появление всех четырёх типов комнат (бой, сундук, отдых, пусто) с ожидаемой частотой. Проверялась логика видимости комнат и переход на новый этаж после 5 комнат.
- Боевая система: корректность расчёта урона с учётом атаки, защиты, случайного разброса и механики уклонения. Тестировались граничные случаи, такие как победа с минимальным здоровьем.
- Вероятность выпадения предметов: проверка, что предметы выпадают с заявленной вероятностью 30% после победы над врагом.

Все протестированные сценарии завершились успешно. Игра стабильно работает в консоли, не содержит критических ошибок или нарушений баланса, которые делали бы прохождение невозможным, и полностью соответствует исходному техническому заданию.

Заключение

В результате выполнения работы был успешно разработан полнофункциональный прототип консольной текстовой RPG. Программа реализует все ключевые механики, заявленные в задании: создание персонажа с системой рас, процедурную генерацию подземелья, тактическую пошаговую боевую систему, прогрессию через уровни и очки улучшения, а также систему инвентаря и экипировки.

Архитектура проекта, построенная на трёх основных классах (`Character`, `Enemy`, `Game`) и вспомогательном классе `Item`, обеспечивает чёткое разделение ответственности. Применение объектно-ориентированного подхода позволило создать логичные и переиспользуемые компоненты.

Особенностью реализации является учёт физических параметров персонажа (рост и вес), влияющих на игровые характеристики, что добавляет дополнительный уровень глубины. Система прогрессии сложности с множителем 30% на этаж обеспечивает плавное возрастание трудности.

Игра предоставляет законченный игровой цикл, сочетающий стратегическое планирование с тактическими решениями и элементами случайности. Проект демонстрирует практическое применение знаний, полученных в ходе изучения дисциплины «Информатика и программирование».